

Go for Senior .NET Engineers

Phase 1: Syntax & Language Fundamentals

15 Detailed Lessons with C# Comparisons

Current as of Go 1.25.6 (January 2026)

For engineers with 5+ years .NET / C# background

Lesson 1: Package System & main Entry Point

In C# you think in namespaces, classes, and static Main methods. Go has none of that. The fundamental unit of organization is the package, and the entry point is brutally simple.

1.1 — The Minimal Go Program

```
package main

import "fmt"

func main() {
    fmt.Println("Hello from Go 1.25")
}
```

TIP: There is no class, no static keyword, no namespace. The file IS the unit. package main + func main() is your entry point.

1.2 — C# vs Go: Entry Point

C# / .NET

```
namespace MyApp;

class Program {
    static void Main(string[] args) {
        Console.WriteLine("Hello");
    }
}
```

Go

```
package main

import "fmt"

func main() {
    fmt.Println("Hello")
}
```

1.3 — Package Rules

Every .go file starts with a package declaration. All files in the same directory must share the same package name. There is no file-per-class convention — you organize by responsibility, not by type.

Exported identifiers start with an uppercase letter. This replaces C#'s public/private/internal modifiers entirely. If it starts with a capital letter, it's public. If lowercase, it's package-private.

```
package mathutil

// Add is exported (uppercase) – visible outside this package
func Add(a, b int) int { return a + b }

// multiply is unexported (lowercase) – package-private
func multiply(a, b int) int { return a * b }
```

WARNING: Go has no 'internal' access modifier, no 'protected', no 'friend assemblies'. It is strictly uppercase = public, lowercase = package-scoped.

1.4 — Package Layout (Go 1.25 Convention)

```
myproject/
├── go.mod           // Module definition (like .csproj)
├── main.go          // package main – entry point
├── handler/
│   └── user.go      // package handler
├── model/
│   └── user.go      // package model
├── repository/
│   └── user_repo.go // package repository
```

Unlike .NET solutions with multiple projects, a Go module is a single unit defined by `go.mod`. Packages are simply directories. No `.csproj`, no `.sln`, no Assembly references.

1.5 — Imports

```
import (
    "fmt"                // standard library
    "net/http"           // nested std package
    "github.com/gin-gonic/gin" // third-party module
    myalias "myproject/internal/util" // aliased import
    _ "github.com/lib/pq"    // blank import (side-effect only)
)
```

TIP: Unused imports are compile errors in Go. The blank identifier `_` imports a package only for its `init()` side effects, like registering a database driver.

Lesson 2: Variable Declaration — :=, var, Zero Values

Coming from C#, you're used to var with type inference and default(T). Go has similar concepts but with different syntax and one critical philosophical difference: there is no null by default.

2.1 — Declaration Styles

```
// Full declaration (like C# explicit typing)
var name string = "Gopher"

// Type inferred (like C# var)
var age = 30

// Short declaration (Go-specific, most common)
city := "Tashkent"

// Multiple declarations
var (
    host    string = "localhost"
    port    int    = 8080
    debug   bool   // zero value: false
)
```

2.2 — Zero Values (No null Surprise)

Every type in Go has a zero value. There is no uninitialized memory. This is like default(T) in C#, but it applies universally and there is no null reference exception for value types.

Type	Zero Value
int, int64, float64	0
string	"" (empty string)
bool	false
pointer, slice, map, chan, func, interface	nil
struct	all fields zeroed recursively
array	all elements zeroed

2.3 — C# vs Go: Variable Declaration

C# / .NET

```
string name = "Go";
var age = 30;           // inferred
int count;             // 0 (value type)
string? s = null;      // nullable ref
```

Go

```
var name string = "Go"
var age = 30           // inferred
var count int          // 0 (zero value)
var s *string = nil    // pointer,
explicit
```

```
// No equivalent of :=
```

```
city := "Tashkent"    // short  
declaration
```

WARNING: := can only be used inside functions. At package level, you must use var. This trips up every C# dev on day one.

2.4 — Constants

```
const Pi = 3.14159           // untyped constant  
const MaxRetries int = 3     // typed constant  
  
// iota - Go's enum-like auto-incrementor  
const (  
    StatusPending = iota    // 0  
    StatusActive   // 1  
    StatusClosed   // 2  
)  
  
// Bit flags with iota  
const (  
    FlagRead    = 1 << iota // 1  
    FlagWrite   // 2  
    FlagExecute // 4  
)
```

TIP: Go has no enum keyword. iota inside const blocks is how you create enumerations. It resets to 0 in each new const block.

Lesson 3: Basic Types, Type Conversions

Go's type system is simpler than C#'s. No implicit conversions, no boxing/unboxing, and the numeric types are explicit about their size.

3.1 — Numeric Types

Go Type	C# Equivalent
int (platform-dependent: 32/64)	int (always 32-bit) / nint
int8, int16, int32, int64	sbyte, short, int, long
uint8 (= byte), uint16, uint32, uint64	byte, ushort, uint, ulong
float32, float64	float, double
complex64, complex128	System.Numerics.Complex
rune (= int32)	char (= uint16)
bool	bool
string	string (but immutable byte slice)

3.2 — No Implicit Conversions

```
var i int = 42
var f float64 = float64(i)    // explicit cast required
var u uint = uint(f)          // explicit again

// This will NOT compile:
// var f float64 = i           // ERROR: cannot use int as float64

// Even between int sizes:
var a int32 = 10
var b int64 = int64(a)        // must be explicit
```

WARNING: In C#, int implicitly converts to long or double. In Go, EVERY numeric conversion must be explicit. No exceptions.

3.3 — String and Byte Conversions

```
// String to byte slice
s := "Hello"
b := []byte(s)           // copies the data

// Byte slice to string
s2 := string(b)

// int to string — DOES NOT work like C# .ToString()
// string(65) gives "A" (rune conversion), not "65"
```

```
// Use strconv for number-to-string:
import "strconv"
numStr := strconv.Itoa(65)    // "65"
parsed, err := strconv.Atoi("42") // 42, nil
```

TIP: `string(someInt)` converts a rune to its UTF-8 representation, NOT a number to its string form. Use `strconv.Itoa()` for that.

Lesson 4: Functions — Multiple Returns, Named Returns, Variadic

Functions in Go are first-class citizens (like in C#), but Go functions have features that will change how you write code: multiple return values are idiomatic and used everywhere, especially for error handling.

4.1 — Basic Function Syntax

C# / .NET

```
// C#
int Add(int a, int b) {
    return a + b;
}

// With tuple return
(int sum, int product) Calculate(int a,
int b) {
    return (a + b, a * b);
}
```

Go

```
// Go
func Add(a, b int) int {
    return a + b
}

// Multiple return values (native)
func Calculate(a, b int) (int, int) {
    return a + b, a * b
}
```

4.2 — The (value, error) Pattern

This is the most important Go idiom. Instead of throwing exceptions, Go functions return an error as the last return value. You will see this thousands of times.

```
func Divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division by zero")
    }
    return a / b, nil
}

// Caller MUST handle the error
result, err := Divide(10, 0)
if err != nil {
    log.Fatal(err)
}
fmt.Println(result)
```

4.3 — Named Return Values

```
func ParseConfig(path string) (config Config, err error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return // returns zero Config and the error
    }
    err = json.Unmarshal(data, &config)
    return // returns config and err (named returns)
```

```
}
```

TIP: Named returns act as pre-declared variables. A bare 'return' returns them all. Use sparingly — they can hurt readability in long functions.

4.4 — Variadic Functions

```
// Like C# params keyword
func Sum(nums ...int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}

// Usage
Sum(1, 2, 3)           // direct args
nums := []int{1, 2, 3}
Sum(nums...)           // spread a slice
```

4.5 — Functions as Values

```
// Assign function to variable
square := func(n int) int { return n * n }
fmt.Println(square(5)) // 25

// Higher-order function
func Apply(f func(int) int, val int) int {
    return f(val)
}
result := Apply(square, 7) // 49
```

Lesson 5: Control Flow — if, switch, for

Go simplifies control flow to three constructs. There is no while, no do-while, no foreach keyword. The for loop handles everything. switch is more powerful than C# and does not fall through by default.

5.1 — if with Init Statement

```
// Go allows an init statement before the condition
if err := doSomething(); err != nil {
    log.Fatal(err)
}
// err is scoped to this if-else block only

// C# equivalent (less elegant):
// var err = DoSomething();
// if (err != null) { ... }
```

5.2 — switch (No Fall-Through)

C# / .NET

```
// C# – needs break
switch (day) {
    case "Mon":
    case "Tue":
        Console.WriteLine("Early week");
        break; // required!
    default:
        Console.WriteLine("Other");
        break;
}
```

Go

```
// Go – no fall-through by default
switch day {
case "Mon", "Tue":
    fmt.Println("Early week")
    // no break needed
default:
    fmt.Println("Other")
}
```

5.3 — switch Without a Condition

```
// Acts like if-else chain – very idiomatic Go
switch {
case temp < 0:
    fmt.Println("Freezing")
case temp < 20:
    fmt.Println("Cool")
case temp < 35:
    fmt.Println("Warm")
default:
    fmt.Println("Hot")
}
```

5.4 — for as the Only Loop

```
// Traditional for loop
```

```
for i := 0; i < 10; i++ {
    fmt.Println(i)
}

// While-style (no while keyword exists)
for count > 0 {
    count--
}

// Infinite loop
for {
    // break when done
}

// Range over slice (like C# foreach)
for index, value := range items {
    fmt.Printf("%d: %s\n", index, value)
}

// Range over integer (Go 1.22+)
for i := range 10 {
    fmt.Println(i) // 0 through 9
}
```

TIP: for i := range 10 was added in Go 1.22 and is now the cleanest way to loop N times.

Lesson 6: Pointers — Explicit but Safe

In C# you rarely think about pointers because reference types handle indirection for you. In Go, there are no reference types vs value types in the C# sense. Everything is a value. If you want indirection, you use a pointer explicitly.

6.1 — Pointer Basics

```
x := 42
p := &x          // p is *int, holds address of x
fmt.Println(*p)  // 42 – dereference the pointer
*p = 100         // modify x through the pointer
fmt.Println(x)   // 100
```

6.2 — C# Reference vs Go Pointer

C# / .NET

```
// C# – class is reference type
class User { public string Name; }

void Modify(User u) {
    u.Name = "Changed";
    // modifies original (reference)
}
```

Go

```
// Go – struct is value type, always
type User struct { Name string }

func Modify(u *User) {
    u.Name = "Changed"
    // explicit pointer required
}
```

6.3 — No Pointer Arithmetic

Unlike C/C++, Go pointers cannot be incremented, decremented, or used for arithmetic. They are safe references. The only operations are & (address-of) and * (dereference).

```
// This will NOT compile:
// p++
// p + 1
// *(p + offset)

// Go is memory-safe by design
// The unsafe package exists but is strongly discouraged
```

6.4 — new() vs &T{}

```
// Both create a pointer to a zero-valued struct
u1 := new(User)          // returns *User (all fields zeroed)
u2 := &User{}            // same result, more idiomatic
u3 := &User{Name: "Go"}  // initialized with values

// No constructors in Go – factory functions instead:
func NewUser(name string) *User {
    return &User{Name: name}
}
```

```
}
```

TIP: Go's escape analysis determines if a variable lives on the stack or heap. You don't decide. Returning `&localVar` is perfectly safe — the compiler heap-allocates it.

Lesson 7: Structs — No Classes, No Inheritance, No Constructors

This is the biggest mental shift. Go has no classes. Structs are the only composite data type. There is no inheritance hierarchy, no base class, no abstract, no virtual. Composition replaces inheritance.

7.1 — Defining a Struct

C#/.NET

```
public class User {  
    public string Name { get; set; }  
    public int Age { get; set; }  
    public string Email { get; set; }  
  
    public User(string name, int age) {  
        Name = name;  
        Age = age;  
    }  
}
```

Go

```
type User struct {  
    Name string  
    Age int  
    Email string  
}  
  
// No constructor, use factory function  
func NewUser(name string, age int) User {  
    return User{Name: name, Age: age}  
}
```

7.2 — Instantiation Patterns

```
// Named fields (preferred)  
u1 := User{Name: "Alice", Age: 30}  
  
// Positional (fragile, avoid)  
u2 := User{"Alice", 30, "alice@go.dev"}  
  
// Zero value struct  
var u3 User // Name: "", Age: 0, Email: ""  
  
// Pointer to struct  
u4 := &User{Name: "Bob"}
```

7.3 — Struct Tags (Metadata)

Struct tags are Go's equivalent of C# attributes on properties, used heavily in JSON serialization, database mapping, and validation.

```
type Product struct {  
    ID int `json:"id" db:"product_id"`  
    Name string `json:"name" validate:"required"`  
    Price float64 `json:"price,omitempty"`  
    CreatedAt time.Time `json:"created_at"`  
}
```

7.4 — Anonymous / Embedded Structs

```
// Embedding (composition, NOT inheritance)
type Address struct {
    City    string
    Country string
}

type Employee struct {
    Name string
    Address      // embedded – fields promoted
}

e := Employee{
    Name: "Alice",
    Address: Address{City: "Tashkent", Country: "UZ"},
}
fmt.Println(e.City) // "Tashkent" – promoted field access
```

TIP: Embedding promotes the inner struct's fields and methods to the outer struct. It looks like inheritance but is composition with syntactic sugar.

Lesson 8: Methods — Receiver Functions

Go doesn't have methods inside a class body. Instead, you define functions with a receiver parameter that associates the method with a type. This is fundamentally different from C#.

8.1 — Value vs Pointer Receivers

```
type Circle struct {
    Radius float64
}

// Value receiver – gets a COPY of Circle
func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

// Pointer receiver – can MODIFY the original Circle
func (c *Circle) Scale(factor float64) {
    c.Radius *= factor
}

// Usage
circle := Circle{Radius: 5}
fmt.Println(circle.Area()) // 78.54...
circle.Scale(2)
fmt.Println(circle.Radius) // 10
```

8.2 — When to Use Which Receiver

Use Pointer Receiver (*T) When	Use Value Receiver (T) When
Method modifies the receiver	Method only reads data
Struct is large (avoid copy cost)	Struct is small (few fields)
Consistency: if any method needs pointer, use pointer for all	Type is a small value like time.Time
The type implements an interface that expects *T	You want immutable semantics

8.3 — C# Methods vs Go Methods

C# / .NET

```
public class Circle {
    public double Radius { get; set; }

    public double Area() {
        return Math.PI * Radius * Radius;
    }
}
```

Go

```
type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}
```

```
public void Scale(double factor) {  
    Radius *= factor;  
}
```

```
}  
  
func (c *Circle) Scale(factor float64) {  
    c.Radius *= factor  
}
```

WARNING: You can only define methods on types declared in the same package. You cannot add methods to types from other packages (no extension methods natively, but see interfaces).

Lesson 9: Interfaces — Implicit Satisfaction (Duck Typing)

This is where Go gets radically different from C#. In C#, a class explicitly implements an interface. In Go, a type satisfies an interface simply by having the right methods. No 'implements' keyword.

9.1 — Defining and Implementing an Interface

C# / .NET

```
// C# – explicit implementation
public interface IWriter {
    void Write(byte[] data);
}

public class FileWriter : IWriter {
    public void Write(byte[] data) {
        // implementation
    }
}
```

Go

```
// Go – implicit satisfaction
type Writer interface {
    Write(data []byte) (int, error)
}

// No "implements" keyword needed
type FileWriter struct { /*...*/ }

func (fw *FileWriter) Write(data []byte)
(int, error) {
    // implementation
    return len(data), nil
}
// FileWriter now satisfies Writer
automatically
```

9.2 — Why Implicit Interfaces Matter

In C#, you need the source code to add an interface. In Go, you can define an interface that existing types already satisfy, without modifying them.

```
// Your own interface
type Stringer interface {
    String() string
}

// Any type with a String() method satisfies this.
// net.IP, time.Time, url.URL all already have String().
// They satisfy YOUR interface without knowing about it.
```

9.3 — The io.Reader / io.Writer Philosophy

Go's standard library is built on tiny interfaces. io.Reader has one method: Read. io.Writer has one method: Write. Everything composes from these building blocks.

```
// The entire io.Reader interface:
type Reader interface {
    Read(p []byte) (n int, err error)
}
```

```
// These all satisfy io.Reader:
// os.File, bytes.Buffer, strings.Reader,
// http.Response.Body, gzip.Reader, crypto/tls.Conn...

// Your function accepts ANY reader:
func CountBytes(r io.Reader) (int64, error) {
    return io.Copy(io.Discard, r)
}
```

TIP: Go proverb: 'The bigger the interface, the weaker the abstraction.' Prefer interfaces with 1-3 methods. This is the opposite of C#'s `IRepository<T>` patterns with 20 methods.

9.4 — Compile-Time Interface Check

```
// Ensure a type satisfies an interface at compile time
var _ Writer = (*FileWriter)(nil)

// If FileWriter doesn't have the Write method,
// this line causes a compile error.
```

Lesson 10: Error Handling — No Exceptions, No try/catch

This is the most controversial aspect of Go for C# developers. There are no exceptions. No try/catch/finally. No throw. Errors are values returned from functions, and you check them explicitly.

10.1 — The error Interface

```
// The built-in error interface:
type error interface {
    Error() string
}

// Creating errors
import "errors"
err := errors.New("something went wrong")

// Formatted error
err := fmt.Errorf("failed to read file %s: %w", path, originalErr)
```

10.2 — The if err != nil Pattern

C# / .NET

```
// C# - exception-based
try {
    var data = File.ReadAllText(path);
    var config = JsonSerializer
        .Deserialize<Config>(data);
    return config;
} catch (FileNotFoundException ex) {
    logger.Error(ex);
    throw;
}
```

Go

```
// Go - error-value-based
data, err := os.ReadFile(path)
if err != nil {
    return Config{}, fmt.Errorf(
        "read config: %w", err)
}
var config Config
if err := json.Unmarshal(data, &config);
err != nil {
    return Config{}, fmt.Errorf(
        "parse config: %w", err)
}
return config, nil
```

10.3 — Error Wrapping (Go 1.13+)

```
// Wrap with %w to preserve the error chain
if err != nil {
    return fmt.Errorf("open database: %w", err)
}

// Unwrap and check
if errors.Is(err, os.ErrNotExist) {
    // handle file not found
}
```

```
// Type assertion on error
var pathErr *os.PathError
if errors.As(err, &pathErr) {
    fmt.Println("Failed path:", pathErr.Path)
}
```

10.4 — Custom Error Types

```
type ValidationError struct {
    Field   string
    Message string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("validation: %s - %s", e.Field, e.Message)
}

// Usage
func Validate(u User) error {
    if u.Name == "" {
        return &ValidationError{Field: "Name", Message: "required"}
    }
    return nil
}
```

WARNING: Yes, the repetitive 'if err != nil' feels verbose. This is by design. Go forces you to handle every error at the call site. Resist the urge to create generic error-handling wrappers — embrace the explicitness.

Lesson 11: Arrays vs Slices vs Maps

Coming from C#, you used `List<T>`, `Dictionary<K,V>`, and arrays. Go's collection primitives are different: arrays are fixed-size value types, slices are dynamic views over arrays, and maps are built-in hash tables.

11.1 — Arrays (Rarely Used Directly)

```
// Arrays are fixed-size VALUE types (copied on assignment!)
var a [5]int           // [0, 0, 0, 0, 0]
b := [3]string{"Go", "C#", "Rust"}
c := [...]int{1, 2, 3, 4} // size inferred as 4

// Arrays are values – this creates a COPY:
x := [3]int{1, 2, 3}
y := x                // y is a full copy
y[0] = 99
fmt.Println(x[0])     // 1 (unchanged)
```

11.2 — Slices (The Go `List<T>`)

```
// Slice creation
s := []int{1, 2, 3}           // literal
s2 := make([]int, 5)          // len=5, cap=5, all zeros
s3 := make([]int, 0, 10)      // len=0, cap=10 (pre-allocate)

// Append (like List<T>.Add)
s = append(s, 4, 5)
s = append(s, otherSlice...) // spread

// Slicing (creates a VIEW, not a copy)
sub := s[1:3]                 // elements at index 1, 2
```

11.3 — Slice Internals (Critical Knowledge)

A slice is a struct with three fields: pointer (to underlying array), length, and capacity. Understanding this prevents subtle bugs.

```
// Slice header (conceptually):
// type slice struct {
//     ptr *array
//     len int
//     cap int
// }

original := make([]int, 3, 5)
sub := original[1:3]
// sub shares the same underlying array!
sub[0] = 99
fmt.Println(original[1]) // 99 <-- modified!
```

```
// Use slices.Clone (Go 1.21+) for a true copy:
import "slices"
safe := slices.Clone(original)
```

11.4 — Maps

```
// Map creation
m := map[string]int{
    "Go":      2009,
    "CSharp": 2000,
    "Rust":    2010,
}

// Or with make
m2 := make(map[string]int)

// Access with comma-ok idiom
year, ok := m["Go"]
if !ok {
    fmt.Println("key not found")
}

// Delete
delete(m, "Rust")

// Iterate (order is random!)
for key, value := range m {
    fmt.Printf("%s: %d\n", key, value)
}
```

WARNING: Map iteration order is randomized in Go (intentionally). Never depend on key order. Go 1.24 uses a Swiss Table implementation internally, making maps 2-3% faster.

Lesson 12: Strings, Runes, and UTF-8

In C#, a string is a sequence of UTF-16 chars (each 2 bytes). In Go, a string is a read-only sequence of bytes, typically UTF-8 encoded. This difference affects indexing, length, and iteration.

12.1 — String Fundamentals

```
s := "Hello, World!"

// len() returns BYTE count, not character count
fmt.Println(len(s))           // 13

// For a Cyrillic/CJK/emoji string:
ru := "Привет"
fmt.Println(len(ru))           // 12 (bytes, not 6 chars)

// Character count:
import "unicode/utf8"
fmt.Println(utf8.RuneCountInString(ru)) // 6
```

12.2 — Runes (Go's char)

A rune is an alias for int32 and represents a Unicode code point. It's Go's equivalent of C#'s char, but it handles the full Unicode range without surrogate pairs.

```
// Indexing gives bytes, not runes!
s := "café"
fmt.Println(s[3])           // 195 (first byte of é)

// Range loop iterates over RUNES automatically:
for i, r := range "café" {
    fmt.Printf("index=%d rune=%c byte_pos=%d\n", i, r, i)
}
// index=0 rune=c byte_pos=0
// index=1 rune=a byte_pos=1
// index=2 rune=f byte_pos=2
// index=3 rune=é byte_pos=3 (é is 2 bytes)
```

12.3 — String Builder (Efficient Concatenation)

```
// Like C# StringBuilder
var sb strings.Builder
for i := 0; i < 1000; i++ {
    sb.WriteString("item ")
    sb.WriteString(strconv.Itoa(i))
    sb.WriteByte('\n')
}
result := sb.String()
```

12.4 — Common String Operations

C#	Go
<code>s.Contains("x")</code>	<code>strings.Contains(s, "x")</code>
<code>s.Split(",")</code>	<code>strings.Split(s, ",")</code>
<code>s.Trim()</code>	<code>strings.TrimSpace(s)</code>
<code>s.Replace("a","b")</code>	<code>strings.ReplaceAll(s, "a", "b")</code>
<code>s.ToUpper()</code>	<code>strings.ToUpper(s)</code>
<code>\$"Hello {name}"</code>	<code>fmt.Sprintf("Hello %s", name)</code>
<code>s.StartsWith("x")</code>	<code>strings.HasPrefix(s, "x")</code>

TIP: Go has no string interpolation syntax. Use `fmt.Sprintf()` with format verbs: `%s` (string), `%d` (int), `%f` (float), `%v` (default), `%+v` (with field names), `%#v` (Go syntax).

Lesson 13: defer, panic, recover

These three form Go's resource cleanup and recovery mechanism, replacing C#'s try/catch/finally and using/IDisposable patterns.

13.1 — defer (Go's finally + using)

```
// defer schedules a function call to run when the
// surrounding function returns (LIFO order)

func ReadFile(path string) ([]byte, error) {
    f, err := os.Open(path)
    if err != nil {
        return nil, err
    }
    defer f.Close() // guaranteed cleanup

    return io.ReadAll(f)
}
```

13.2 — C# using vs Go defer

C# / .NET

```
// C# - IDisposable + using
using var conn = new SqlConnection(cs);
conn.Open();
// auto-disposed at end of scope

// Or try/finally:
var conn = new SqlConnection(cs);
try {
    conn.Open();
} finally {
    conn.Close();
}
```

Go

```
// Go - defer
conn, err := sql.Open("postgres", cs)
if err != nil {
    return err
}
defer conn.Close()

// conn.Close() runs when this
// function returns, regardless
// of how it returns
```

13.3 — defer LIFO Order

```
func main() {
    defer fmt.Println("First deferred")
    defer fmt.Println("Second deferred")
    defer fmt.Println("Third deferred")
    fmt.Println("Running...")
}

// Output:
// Running...
// Third deferred ← LIFO (stack order)
// Second deferred
// First deferred
```

13.4 — panic and recover

panic is Go's equivalent of throwing an unhandled exception. It unwinds the stack. recover can catch a panic inside a deferred function — this is the closest thing to try/catch.

```
// panic - terminate with a message
func MustParse(s string) int {
    n, err := strconv.Atoi(s)
    if err != nil {
        panic(fmt.Sprintf("invalid number: %s", s))
    }
    return n
}

// recover - catch a panic (only in defer)
func SafeOperation() (err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("recovered: %v", r)
        }
    }()

    riskyFunction()
    return nil
}
```

WARNING: Go convention: don't use panic for normal error handling. Panics are for truly unrecoverable situations (nil pointer, index out of range, programmer bugs). Use error returns for expected failures.

Lesson 14: Type Embedding (Composition Over Inheritance)

Go enforces composition. There is no extends, no base class, no virtual override. Type embedding provides delegation and field promotion, achieving polymorphism through interfaces.

14.1 — Embedding a Struct

```
type Logger struct {}

func (l Logger) Log(msg string) {
    fmt.Printf("[LOG] %s\n", msg)
}

type Server struct {
    Logger          // embedded – Server "has a" Logger
    Host    string
    Port    int
}

func main() {
    s := Server{Host: "localhost", Port: 8080}
    s.Log("Starting...") // promoted from Logger
    // equivalent to s.Logger.Log("Starting...")
}
```

14.2 — Embedding an Interface

```
// Embed interfaces to compose them
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Writer interface {
    Write(p []byte) (n int, err error)
}

// Composed interface
type ReadWriter interface {
    Reader
    Writer
}

// Any type with both Read and Write satisfies ReadWriter
```

14.3 — C# Inheritance vs Go Composition

C#/.NET

```
// C# – inheritance chain
public abstract class Animal {
```

Go

```
// Go – interfaces + composition
type Speaker interface {
```

```
public string Name { get; set; }  
public abstract string Speak();  
}  
  
public class Dog : Animal {  
    public override string Speak()  
        => "Woof!";  
}
```

```
Speak() string  
}  
  
type Animal struct {  
    Name string  
}  
  
type Dog struct {  
    Animal // embedded  
}  
  
func (d Dog) Speak() string {  
    return "Woof!"  
}
```

TIP: The Go proverb is: 'Don't design with interfaces, discover them.' Write concrete types first, then extract interfaces only when you need polymorphism (e.g., for testing or multiple implementations).

Lesson 15: Blank Identifier `_`, `init()`, Package Initialization

These are Go-specific mechanisms for handling unused values and controlling startup behavior. They have no direct C# equivalent.

15.1 — The Blank Identifier `_`

```
// Discard a return value (compiler requires all vars to be used)
result, _ := strconv.Atoi("42") // ignore the error

// Skip index in range
for _, item := range items {
    process(item)
}

// Import for side effects only
import _ "github.com/lib/pq" // registers PostgreSQL driver

// Compile-time interface check
var _ io.Writer = (*MyWriter)(nil)
```

WARNING: Ignoring errors with `_` is legal but dangerous. Linters like `golangci-lint` will flag this. Always handle errors in production code.

15.2 — `init()` Functions

```
package config

var defaults map[string]string

// init runs automatically before main()
// A file can have multiple init() functions
func init() {
    defaults = map[string]string{
        "host": "localhost",
        "port": "8080",
    }
    fmt.Println("config package initialized")
}
```

15.3 — Package Initialization Order

Go initializes packages in dependency order. Within a package: package-level variables are initialized first (in declaration order), then `init()` functions run, then `main()` runs.

```
// Initialization order:
// 1. Imported packages (depth-first)
// 2. Package-level variables (in source order)
// 3. init() functions (in source order, per file)
```

```
// 4. main() function

var (                                // Step 2
    dbURL  = os.Getenv("DB_URL")
    debug  = os.Getenv("DEBUG") == "true"
)

func init() {                        // Step 3
    if dbURL == "" {
        dbURL = "localhost:5432"
    }
}

func main() {                        // Step 4
    fmt.Println("DB:", dbURL)
}
```

TIP: Modern Go style discourages `init()`. Prefer explicit initialization in `main()`. `init()` makes code harder to test and reason about. Use it sparingly for truly global setup like registering drivers.

15.4 — Phase 1 Summary: Key Mindset Shifts

C# / .NET Concept	Go Equivalent / Replacement
namespace + class	package + functions
public / private / internal	Uppercase / lowercase first letter
null	nil (only for pointers, interfaces, slices, maps, channels, funcs)
var with type inference	<code>:=</code> short declaration
implicit conversions	explicit conversions everywhere
class + constructor	struct + factory function (<code>NewXxx</code>)
class methods	receiver functions (value or pointer)
interface <code>IFoo : IBar</code>	implicit satisfaction (duck typing)
try / catch / finally	<code>if err != nil + defer</code>
<code>List<T></code>	<code>[]T</code> (slice)
<code>Dictionary<K,V></code>	<code>map[K]V</code>
string (UTF-16)	string (UTF-8 byte sequence)
using / <code>IDisposable</code>	<code>defer</code>
base class inheritance	struct embedding (composition)
throw new <code>Exception()</code>	return error / panic (rare)